Listing 11.12 Creating an authentication handler based on a key

```
# RiffMates/api_auth.py
from django.conf import settings

from ninja.security import APIKeyHeader

class APIKey(APIKeyHeader):
    param_name = "X-API-KEY"

    def authenticate(self, request, key):
        if key == settings.NINJA_API_KEY:
            return key

api_key = APIKey()
```

The name of the HTTP header where the key gets found

Compare the key sent in to a hardcoded value in settings.py.

Return the key if the values match; otherwise, use the default return value of a function: None.

Create a single instance of APIKey to be used by the apps.

Don't forget to add a new configuration value, called `NINJA_API_KEY`, in your settings.py file. I've set mine to the very cryptic *notsecure*. As mentioned earlier, this is a bad idea. Do proper key management, or at least use an appropriately cryptic long value, for your key.

11.2.6 Creating objects with the API

With your key-handling class in place, you can now write an endpoint for creating a venue that is behind the authentication wall. You may have noticed that I named the serializer for venues `VenueOut`. That's because there needs to be a `VenueIn`. The fields involved in displaying a venue are different from those for creating one. Venues being output should include their ID, whereas venues getting created don't have an ID yet. In this case, the only difference between the *in* and *out* versions is the ID, so you could choose an ID value of 0 for creating, but in more complex cases, there may be bigger differences.

When using the `.post()` decorator for the creation method, you declare an argument to the function for the field values. This is similar to the contents of POST when handling a form, but in this case, the payload is a JSON dictionary. You tell Ninja what kind of data to expect by using a Python type hint for the payload argument. The `Schema` class has a `.dict()` method that returns the contents as a dictionary. This can be combined with Python's double-star (`**`) feature, like you do with `**kwargs`, to use the contents of the dictionary as key–value pairs for the `Venue` object constructor. The following listing contains the new `VenueIn` serializer along with the POST endpoint for creating a venue.

Listing 11.13 A create endpoint for venues

```
# RiffMates/bands/api.py
...
from api_auth import api_key
```

Import the instance of APIKey to specify the authentication method.